NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 5: Tuples, Lists, and Ordered Engineering Data

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 5 at a glance

Lecture 13

Ordered collections
Lists, indices, slicing, loops, and summaries.

Lecture 14

Mutability and operations
Changing, building, filtering, sorting, aliasing, and copying.

Lecture 15

Tuples and records
Immutability, unpacking, paired data, and Laboratory 2.

Store → Access → Change deliberately → Preserve meaning and units

NAZARBAYEV UNIVERSITY

LECTURE 13

Ordered Collections, Indexing, and Slicing

Moving from one engineering value to an ordered sequence of measurements, design cases, or states

NAZARBAYEV UNIVERSITY

Lecture 13 learning goals

By the end of this lecture, you should be able to:

1. explain why engineers need ordered collections;
2. create and inspect a Python list;
3. distinguish an element's value from its index;
4. use zero-based and negative indexing;
5. identify valid indices and diagnose `IndexError`;
6. extract data windows using slicing;
7. use `len()`, `sum()`, `min()`, and `max()`;
8. process a list by value, by index, and with `enumerate()`;
9. preserve order, physical meaning, and units;
10. trace and repair short collection programs.

Order converts several values into interpretable engineering data

NAZARBAYEV UNIVERSITY

Warm-up: one sensor, several measurements

A temperature sensor records once per second

Time (s)	0	1	2	3	4
Temperature (°C)	22.1	22.5	23.0	23.4	22.9

Discuss

Why is one variable insufficient?
What does the order preserve?
Which summaries might support an engineering decision?

Pairs - 3 min

NAZARBAYEV UNIVERSITY

Engineering data is usually more than one value

A scalar variable

One force, one area, one stress, or one temperature at a particular state.

An ordered collection

Sensor readings over time, candidate dimensions, forces during a test, or states generated by a simulation.

Representation is part of problem solving

The program must preserve which values belong together and what their order means physically.

A first Python list

```
temperature_readings_c = [
    22.1, 22.5, 23.0, 23.4, 22.9
]
print(temperature_readings_c)
print(type(temperature_readings_c))
```

Output

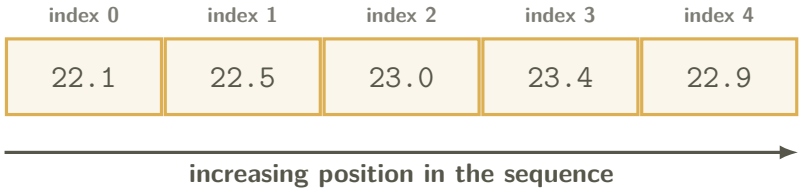
```
[22.1, 22.5, 23.0, 23.4, 22.9]
<class 'list'>
```

Syntax

Square brackets enclose comma-separated elements.

NAZARBAYEV UNIVERSITY

A list is ordered



Order carries meaning

Index 0 is the first recorded value. Reordering a time series or design sequence can change its interpretation.

Indexing retrieves one element

```
readings = [22.1, 22.5, 23.0, 23.4, 22.9]
first = readings[0]
second = readings[1]
fourth = readings[3]
print(first, second, fourth)
```

Output

```
22.1 22.5 23.4
```

Zero-based indexing

The first element is at index 0, not index 1.

NAZARBAYEV UNIVERSITY

Index is not the same as value

Index

An integer position that answers "Where is the element?"

`readings[2]`

Value

The object stored at that position. Here the result is

`23.0`

Read aloud

`readings[2]` means "the element of `readings` at index 2."

NAZARBAYEV UNIVERSITY

Negative indices count from the end

```
readings = [22.1, 22.5, 23.0, 23.4, 22.9]
last = readings[-1]
second_last = readings[-2]
print(last, second_last)
```

Useful patterns

`x[-1]` is last; `x[-2]` is second-to-last.

Engineering use

Access the latest reading without calculating its positive index.

NAZARBAYEV UNIVERSITY

Valid indices for a list of length n

Positive indices
$0, 1, \ldots, n-1$

Negative indices
$-1, -2, \ldots, -n$

Classic off-by-one error

If `len(values) == n`, then `values[n]` is outside the list. The last positive index is `n - 1`.

NAZARBAYEV UNIVERSITY

An invalid index raises extttIndexError

```
values = [10, 20, 30]
print(len(values))
print(values[3])
```

Typical output

```
3
IndexError: list index out of
    range
```

Diagnosis

Valid indices are 0, 1, and 2.

NAZARBAYEV UNIVERSITY

Slicing retrieves a window of data

```
readings = [22.1, 22.5, 23.0, 23.4, 22.9]
window = readings[1:4]
print(window)
```

Output

```
[22.5, 23.0, 23.4]
```

Rule

`x[start:stop]` includes start and excludes stop.

Start included; stop excluded

NAZARBAYEV UNIVERSITY

Why is the stopping index excluded?

Predictable length

With step 1, `x[a:b]` contains $b - a$ elements.

Adjacent windows

`x[0:3]` and `x[3:6]` do not overlap and leave no gap.

Engineering use

Partition calibration, test, and validation windows with explicit non-overlapping boundaries.

Useful slice patterns

```
time_s = [0, 1, 2, 3, 4, 5, 6]
first_three = time_s[:3]
from_two = time_s[2:]
every_second = time_s[::2]
backward = time_s[::-1]
```

Meaning

`x[:b]` begins at 0; `x[a:]` continues to the end; `x[::k]` uses step k; `x[::-1]` reverses order.

NAZARBAYEV UNIVERSITY

Prediction activity: index or slice?

```
data = [5, 10, 15, 20, 25, 30]
A = data[2]
B = data[-2]
C = data[1:4]
D = data[:3]
E = data[::2]
F = data[4:1:-1]
```

Before running

Predict A–F. Mark which expressions return one value and which return a new list. For each slice, mark the included indices.

Individual → pairs - 4 min

NAZARBAYEV UNIVERSITY

Length and membership answer structural questions

```
standard_diameters_mm = [6, 8, 10, 12, 16]
number_available = len(standard_diameters_mm)
has_10_mm = 10 in standard_diameters_mm
has_14_mm = 14 in standard_diameters_mm
print(number_available, has_10_mm, has_14_mm)
```

Output

```
5 True False
```

Questions

How many options exist? Is the required standard size available?

NAZARBAYEV UNIVERSITY

Simple summaries with built-in functions

```
readings_c = [22.1, 22.5, 23.0, 23.4, 22.9]
average_c = sum(readings_c) / len(readings_c)
minimum_c = min(readings_c)
maximum_c = max(readings_c)
range_c = maximum_c - minimum_c
print(f"Average = {average_c:.2f} deg C")
print(f"Range = {range_c:.2f} deg C")
```

Interpret carefully

A mean does not reveal order, transients, outliers, or measurement quality.

Empty list

The average formula divides by zero if no readings are present.

Loop directly over values

```
pressures_kpa = [95, 102, 108, 97]
warning_limit_kpa = 100
for pressure_kpa in pressures_kpa:
    if pressure_kpa > warning_limit_kpa:
        print(f"Warning: {pressure_kpa} kPa")
```

Use when

The calculation needs each element but not its position.

Naming

Name the loop variable after one element: `pressure_kpa`, not `x`.

NAZARBAYEV UNIVERSITY

Loop over indices when position matters

```
time_s = [0, 1, 2, 3]
temperature_c = [20.0, 21.8, 23.5, 25.1]
for i in range(len(time_s)):
    print(f"t = {time_s[i]} s, "
          f"T = {temperature_c[i]:.1f} deg C")
```

Use when

Two lists must be accessed at the same position, the index has meaning, or an element will be changed.

Precondition

Paired lists must have compatible lengths and matching order.

NAZARBAYEV UNIVERSITY

extttenumerate() gives index and value together

```
readings_c = [22.1, 22.5, 23.0]
for index, reading_c in enumerate(readings_c):
    print(f"Sample {index}: {reading_c:.1f} deg
    C")
```

Output

```
Sample 0: 22.1 deg C
Sample 1: 22.5 deg C
Sample 2: 23.0 deg C
```

Advantage

Both position and value are visible without repeated indexing.

NAZARBAYEV UNIVERSITY

Three loop patterns – choose deliberately

Pattern	Best when	Typical use
`for value in values`	only element matters	sum, threshold, count
`for i in range(len(values))`	position controls access/change	paired lists, replacement
`for i, value in enumerate(values)`	position and value both matter	labeled report, event location

Readability principle

Prefer the simplest loop that exposes the information your algorithm needs.

NAZARBAYEV UNIVERSITY

Trace: find the first threshold crossing

```
temperature_c = [20.0, 24.5, 29.2, 35.1, 41.0]
limit_c = 30.0
first_crossing = None
for i, value_c in enumerate(temperature_c):
    if value_c >= limit_c:
        first_crossing = i
        break
print(first_crossing)
```

Before running

Which indices are visited? Where does the loop stop? What would `None` mean after the loop?

Trace - 3 min

NAZARBAYEV UNIVERSITY

Units and meaning do not live automatically inside a list

Ambiguous

```
readings = [22.1, 22.5, 23.0]
```

Temperature? Voltage? Pressure? Which unit?

Meaning visible

```
temperature_readings_c
```

Quantity and unit are visible in the name.

Additional safeguards

Use comments, prompts, table headings, axis labels, and printed reports to preserve units and order.

NAZARBAYEV UNIVERSITY

Debugging clinic: three collection mistakes

```
readings = [2.1, 2.3, 2.2]
# A
print(readings[3])
# B
average = sum(readings) / 4
# C
for i in range(len(readings) + 1):
    print(readings[i])
```

Repair and explain

Which lines raise an error? Which line runs but is wrong? Identify the index or length misconception in each case.

Pairs - 5 min

Lecture 13 exit ticket

Given `values = [4, 7, 9, 12, 15]`

Write the last valid positive index, `values[-2]`, `values[1:4]`, and a loop header that provides both index and value.

Reflection

Why can changing the order of measurements change engineering meaning even when the same numbers remain present?

Submit individually

NAZARBAYEV UNIVERSITY

LECTURE 14

List Mutability and List Operations

Changing, filtering, sorting, and copying collections without corrupting engineering data

NAZARBAYEV UNIVERSITY

Lecture 14 learning goals

1. explain list mutability;
2. replace elements by index;
3. update many elements through a loop;
4. build and filter lists using `append()`;
5. distinguish `append()` from `extend()`;
6. use `insert()`, `remove()`, and `pop()`;
7. distinguish in-place mutation from a returned new list;
8. avoid deletion while iterating;
9. explain aliasing and create an independent copy;
10. recognize side effects through function arguments;
11. preserve raw engineering data;
12. diagnose common list-operation bugs.

Mutable data is powerful only when every change is intentional

NAZARBAYEV UNIVERSITY

Warm-up: predict the final list

```
values = [10, 20, 30]
values[1] = 25
values.append(40)
values.insert(0, 5)
removed = values.pop()
print(values)
print(removed)
```

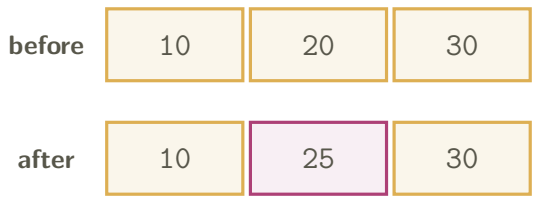
Trace on paper

After each statement, write the complete list, its length, and any value returned by the operation.

Individual - 3 min

NAZARBAYEV UNIVERSITY

Mutable means the object can change after creation



The same list object now contains a different element at index 1.

Engineering implication

Calibration and cleaning can change data directly; accidental mutation can also destroy the original record.

Replace one confirmed element

```
readings_c = [22.1, 22.5, 230.0, 23.4]
# Correct a confirmed data-entry mistake
readings_c[2] = 23.0
print(readings_c)
```

Output

```
[22.1, 22.5, 23.0, 23.4]
```

Evidence

Know which index is wrong and why the replacement is justified.

NAZARBAYEV UNIVERSITY

Apply a calibration correction in place

```
readings_c = [22.1, 22.5, 23.0, 23.4]
offset_c = 0.3
for i in range(len(readings_c)):
    readings_c[i] = readings_c[i] - offset_c
print(readings_c)
```

Why indices?

The loop assigns a new value back to every position.

Risk

The raw readings are overwritten.

NAZARBAYEV UNIVERSITY

Safer calibration: build a processed list

```
raw_readings_c = [22.1, 22.5, 23.0, 23.4]
offset_c = 0.3
corrected_readings_c = []
for raw_c in raw_readings_c:
    corrected_readings_c.append(raw_c - offset_c)
```

Preserve raw data unless replacement is intentional and justified

Traceability

Raw data remain available; the correction is visible; original and processed values can be compared.

NAZARBAYEV UNIVERSITY

Build and filter with extttappend()

```
all_pressures_kpa = [95, 108, 99, 115, 102]
limit_kpa = 105
safe_pressures_kpa = []
for pressure_kpa in all_pressures_kpa:
    if pressure_kpa <= limit_kpa:
        safe_pressures_kpa.append(pressure_kpa)
print(safe_pressures_kpa)
```

Output

```
[95, 99, 102]
```

Pattern

Start empty; evaluate each candidate; append values that satisfy the rule.

NAZARBAYEV UNIVERSITY

extttappend() and extttextend() are different

Append one object

```
a = [1, 2, 3]
a.append([4, 5])
# [1, 2, 3, [4, 5]]
```

Extend with elements

```
b = [1, 2, 3]
b.extend([4, 5])
# [1, 2, 3, 4, 5]
```

Ask first

Is the new object one record, or a batch whose members should become separate elements?

NAZARBAYEV UNIVERSITY

Insert an element at a chosen position

```
time_s = [0, 1, 3, 4]
# A measurement at t = 2 s was recovered
time_s.insert(2, 2)
print(time_s)
```

Output

```
[0, 1, 2, 3, 4]
```

Paired data

Insert corresponding values into every aligned sequence at the same index.

NAZARBAYEV UNIVERSITY

Remove by value or by position?

remove(value)

```
readings = [10, -999, 12, -999]
readings.remove(-999)
# removes first match; returns None
```

pop(index)

```
readings = [10, -999, 12, -999]
removed = readings.pop(1)
# removes by position; returns -999
```

Engineering question

Are you removing a known error value, or a verified measurement position?

Check membership before extttremove()

```
readings = [10, 12, 14]
error_code = -999
if error_code in readings:
    readings.remove(error_code)
else:
    print("No error code found")
```

Without the check

`remove()` raises `ValueError` if the value is absent.

Interpretation

Absence of an error code is useful information.

Danger: deleting while iterating over the same list

```
values = [1, 2, 2, 3, 2, 4]
for value in values:
    if value == 2:
        values.remove(value)
print(values)
```

Why values may be skipped

Removal shifts later elements left while the loop's internal position advances.

Beginner rule

Do not change list length while looping directly over that list.

Safe filtering: build a new list

```
values = [1, 2, 2, 3, 2, 4]
clean_values = []
for value in values:
    if value != 2:
        clean_values.append(value)
print(clean_values)
```

Output

```
[1, 3, 4]
```

Benefits

Every element is examined and the original remains available.

NAZARBAYEV UNIVERSITY

Concatenation and repetition create new lists

```
run_1 = [1.2, 1.4, 1.5]
run_2 = [1.3, 1.5, 1.6]
combined = run_1 + run_2
zeros = [0] * 5
```

Important

`list * 2` repeats a sequence; it does not multiply each number by 2.

Meaning

Concatenate experiments only when ordering and metadata remain interpretable.

NAZARBAYEV UNIVERSITY

In-place mutation versus a new result

Operation	Original list	Returned result
`values.append(x)`	changes	None
`values.insert(i,x)`	changes	None
`values.remove(x)`	changes	None
`values.pop(i)`	changes	removed element
`values.sort()`	changes	None
`sorted(values)`	unchanged	new sorted list
`values + other`	unchanged	new combined list
`values.copy()`	unchanged	new shallow copy

Two questions

Did this operation mutate an existing object? What value did the expression return?

NAZARBAYEV UNIVERSITY

extttsort() versus extttsorted()

Sort in place

```
loads_n = [800, 500, 1200]
loads_n.sort()
# original becomes [500, 800, 1200]
```

Create a sorted copy

```
loads_n = [800, 500, 1200]
ranked_n = sorted(loads_n)
# original order is preserved
```

The extttNone surprise

```
loads_n = [800, 500, 1200]
ranked_n = loads_n.sort()
print(loads_n)
print(ranked_n)
```

Output

```
[500, 800, 1200]
None
```

Repair

Use `loads_n.sort()` alone or `ranked_n = sorted(loads_n)`.

NAZARBAYEV UNIVERSITY

Reverse order: mutate or preserve?

In place

```
values = [1, 2, 3, 4]
values.reverse()
```

New reversed list

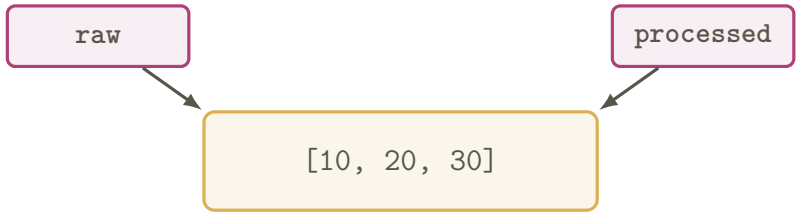
```
values = [1, 2, 3, 4]
backward = values[::-1]
```

Engineering caution

Reversing a time series changes chronological meaning. State the analytical purpose.

NAZARBAYEV UNIVERSITY

Aliasing: two names, one list object



Assignment is not copying

After `processed = raw`, mutation through either name is visible through the other.

Aliasing in code

```
raw = [10, 20, 30]
processed = raw
processed[0] = 999
print(raw)
print(processed)
```

Output

```
[999, 20, 30]
[999, 20, 30]
```

Reason

The assignment created another name, not another list.

NAZARBAYEV UNIVERSITY

Create an independent copy

```
raw = [10, 20, 30]
processed = raw.copy()    # or raw[:]
processed[0] = 999
print(raw)
print(processed)
```

Output

```
[10, 20, 30]
[999, 20, 30]
```

Use

Preserve raw measurements while creating a working dataset.

NAZARBAYEV UNIVERSITY

A function can mutate a supplied list

```
def apply_offset_in_place(readings, offset):
    for i in range(len(readings)):
        readings[i] = readings[i] - offset
raw_c = [22.1, 22.5, 23.0]
apply_offset_in_place(raw_c, 0.3)
print(raw_c)
```

Side effect

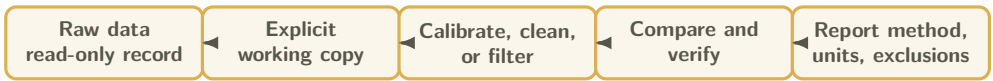
The caller's list changes even though the function returns nothing.

Design

Make mutation explicit in naming/documentation or return a new list.

NAZARBAYEV UNIVERSITY

A safe data-processing workflow



Traceability

Another engineer should be able to reconstruct what was measured, what changed, why, and which data were excluded.

Lecture 14 exit ticket

Explain the difference

`append()` vs `extend()`; `remove()` vs `pop()`; `sort()` vs `sorted()`; aliasing vs copying.

Engineering decision

A laboratory dataset must be calibrated. Would you modify the raw list or create a processed copy? State the evidence to retain.

Submit individually

NAZARBAYEV UNIVERSITY

LECTURE 15

Tuples, Structured Records, and Laboratory 2

Representing fixed ordered information and integrating Week 5 skills in sensor-data processing

NAZARBAYEV UNIVERSITY

Lecture 15 learning goals

1. create empty, singleton, and multi-element tuples;
2. use indexing, slicing, membership, and iteration;
3. explain tuple immutability;
4. pack and unpack values;
5. use multiple assignment and unpack returned values;
6. represent coordinates, bounds, and paired measurements;
7. process a list of tuples using loop unpacking;
8. compare lists and tuples by mutability and purpose;
9. choose a suitable structure for an engineering task;
10. prepare for Laboratory 2.

Use a tuple when a short ordered record should stay fixed

NAZARBAYEV UNIVERSITY

Warm-up: which structure should change?

Choose a structure

Temperature readings over time; one fixed sensor coordinate; a growing set of candidate beam widths.

For each object

Does order matter? Should elements change? Should the collection grow? List or tuple – and why?

Pairs - 4 min

A tuple is an immutable ordered sequence

```
sensor_position_m = (3.0, 4.0)
operating_range_c = (15.0, 35.0)
print(sensor_position_m)
print(type(sensor_position_m))
```

Output

```
(3.0, 4.0)
<class 'tuple'>
```

Properties

Ordered, indexable, iterable, and not modifiable element by element.

The singleton comma matters

```
empty = ()
not_a_tuple = (5)
singleton = (5,)
pair = (5, 8)
print(type(not_a_tuple))
print(type(singleton))
```

Rule

Parentheses can group an expression. The comma creates a one-element tuple: `(5,)`.

NAZARBAYEV UNIVERSITY

Tuples use familiar sequence operations

```
point_m = (3.0, 4.0, 1.5)
x_m = point_m[0]
z_m = point_m[-1]
xy_m = point_m[:2]
print(x_m, z_m, xy_m)
```

Transfer from lists

Zero-based indexing, negative indices, slicing, `len()`, membership, and iteration work similarly.

Immutability prevents item assignment

```
point_m = (3.0, 4.0)
point_m[0] = 5.0
```

Typical message

```
TypeError: 'tuple' object does not support item assignment
```

Purpose

A fixed record cannot be changed accidentally by index assignment.

Why would an engineer want immutability?

Good tuple candidates

A coordinate, lower/upper bounds, one calibration pair, a fixed parameter group, or related function results.

Usually not a tuple

A growing measurement series, frequently corrected values, or a long record whose positions are difficult to remember.

Intent

Immutability communicates that the record's elements should remain fixed after validation.

Packing and unpacking

Packing

```
x_m = 3.0
y_m = 4.0
position_m = x_m, y_m
```

Several values become one tuple.

Unpacking

```
position_m = (3.0, 4.0)
x_m, y_m = position_m
```

One tuple supplies several names.

Contract

The number of target names must match the number of tuple elements.

Multiple assignment can swap values

```
low = 10.0
high = 2.0
if low > high:
    low, high = high, low
print(low, high)
```

Output

```
2.0 10.0
```

Connection

Ordered interval bounds were essential in Week 3 bisection.

NAZARBAYEV UNIVERSITY

Functions can return several related values

```
def summarize(values):
    average = sum(values) / len(values)
    return average, min(values), max(values)
readings = [22.1, 22.5, 23.0, 23.4]
avg, low, high = summarize(readings)
```

Week 4 connection

The return expression is packed into a tuple; the caller stores or unpacks it.

NAZARBAYEV UNIVERSITY

A tuple position needs documented meaning

time (s)	temperature (°C)	uncertainty (°C)
2.4	36.8	0.05

Possible record

```
measurement = (2.4, 36.8, 0.05)
```

Document order

A tuple does not label positions. Use a short stable convention and meaningful unpacking names.

A list of tuples stores paired data

```
calibration_data = [
    (0.0, 0.02),
    (1.0, 1.01),
    (2.0, 2.03),
    (3.0, 3.02)
]
```

Mutable collection of immutable records

Two levels of intent

The list can grow as new points are measured; each (input, output) pair is fixed.

NAZARBAYEV UNIVERSITY

Unpack tuple records directly in a loop

```
calibration_data = [(0.0, 0.02), (1.0, 1.01),
                    (2.0, 2.03)]
for input_v, output_v in calibration_data:
    error_v = output_v - input_v
    print(f"Input = {input_v:.1f} V, "
          f"error = {error_v:+.2f} V")
```

Readable names

The loop exposes the meaning of positions 0 and 1.

Consistency

Every record must have the expected length and order.

Parallel lists versus paired records

Parallel lists

`time_s = [0,1,2]` and `position_m = [0,2,5]`. Insertion or deletion in only one list breaks alignment.

List of tuples

`[(0,0), (1,2), (2,5)]`. Each time-position pair moves as one record while the list preserves order.

Design principle

Store values together when their relationship is more important than separate processing.

NAZARBAYEV UNIVERSITY

Estimate interval velocity from paired records

```
position_data = [(0, 0.0), (1, 2.0),
                 (2, 5.0), (3, 9.0)]
for i in range(1, len(position_data)):
    t0, x0 = position_data[i - 1]
    t1, x1 = position_data[i]
    velocity = (x1 - x0) / (t1 - t0)
    print(f"Interval ending at {t1} s: "
          f"v = {velocity:.1f} m/s")
```

Integrated ideas

Ordered list, fixed tuple records, adjacent indices, unpacking, and an engineering formula.

NAZARBAYEV UNIVERSITY

Lists and tuples share a sequence interface

Operation	List	Tuple
`len(x), x[i], x[a:b]`	yes	yes
`value in x; for value in x`	yes	yes
`x + y; x * n`	yes	yes
`x[i] = value`	yes	no
`append, insert, remove, sort`	yes	no

Transfer

Indexing, slicing, order, membership, and iteration are shared. Mutability is the decisive difference at this stage.

NAZARBAYEV UNIVERSITY

Lists versus tuples: choose by intended use

Choose a list when

Elements will be added, removed, corrected, filtered, sorted, or accumulated during computation.

Choose a tuple when

A short ordered record should stay fixed, several returned values belong together, or immutability communicates a constraint.

Units remain your responsibility

Neither structure automatically stores physical units or labels.

NAZARBAYEV UNIVERSITY

Convert deliberately between list and tuple

```
coordinates_list = [3.0, 4.0]
coordinates_tuple = tuple(coordinates_list)
fixed_bounds = (0.0, 100.0)
working_bounds = list(fixed_bounds)
```

Conversion creates a new object

Use it when the representation requirement changes, such as editable input becoming a fixed validated record.

Structure-selection activity

Choose list or tuple

100 acceleration readings; one 3-D coordinate; standard pipe diameters; fixed temperature bounds; candidate areas; one (time, temperature) record.

Justify with three questions

Must the collection grow or shrink? Must an element be replaced? Does immutability communicate a useful constraint?

Teams - 5 min

NAZARBAYEV UNIVERSITY

Laboratory 2 launch: ordered sensor data

Course-aligned working theme

Process a short acceleration dataset using lists and tuples and produce a clear engineering report.

Dataset

```
[0.12, 0.18, 0.15, 0.22, 0.19,
0.30, 0.28, 0.25, 0.21, 0.17]
```

Unit: $\mathrm{m\,s^{-2}}$

Authoritative instructions

Use Moodle for the final statement, deadline, submission format, collaboration rules, and permitted resources.

Laboratory 2

NAZARBAYEV UNIVERSITY

Laboratory 2: proposed technical tasks

1. store readings in a list;
2. inspect first, last, and first five values;
3. calculate mean, minimum, maximum, and range;
4. calculate RMS with a loop;
5. count readings above a warning limit;
6. preserve raw data and create a corrected copy;
7. build (sample, reading) records;
8. print labels and units;
9. test with hand-checkable values;
10. explain limitations of the summaries.

NAZARBAYEV UNIVERSITY

Laboratory pattern: RMS with a loop

```
readings = [0.12, 0.18, 0.15, 0.22]
sum_of_squares = 0.0
for value in readings:
    sum_of_squares += value**2
mean_square = sum_of_squares / len(readings)
rms = mean_square**0.5
print(f"RMS = {rms:.3f} m/s^2")
```

Model

$$x_{\mathrm{RMS}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

Check

RMS is nonnegative; verify one small case manually.

NAZARBAYEV UNIVERSITY

Debugging clinic: tuple and record mistakes

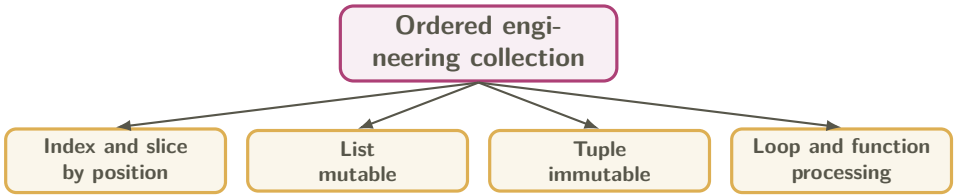
```
# A
point = (3.0)
x, y = point
# B
bounds = (0.0, 100.0)
bounds[1] = 120.0
# C
records = [(0, 20.0), (1, 21.5, "C")]
for time_s, temperature_c in records:
    print(time_s, temperature_c)
# D
average, low = (22.5, 21.0, 24.0)
```

Identify and repair

Singleton syntax, immutability, inconsistent record length, or unpacking mismatch?

Pairs - 5 min

Week 5 concept map and exit ticket



Exit ticket

Design one record for time, voltage, and current, then a collection containing three records. State which part is a list, which part is a tuple, and why.

NAZARBAYEV UNIVERSITY

Week 5 quick reference: sequences

Shared operations

`x[0], x[-1]`	first and last
`x[1:3]`	indices 1 and 2
`x[:3], x[2:]`	open-ended slices
`x[::2], x[::-1]`	step and reverse
`len(x)`	number of elements
`value in x`	membership
`for value in x`	ordered iteration

Choose deliberately

List: ordered and mutable; readings, candidates, and results that grow or change.

Tuple: ordered and immutable; coordinates, bounds, paired records, and related return values.

Index rule

Length n: valid positive indices $0, \ldots, n-1$. Slice start included; stop excluded.

NAZARBAYEV UNIVERSITY

Week 5 quick reference: operations and tuple patterns

List operations

`x[i] = value`	replace element
`x.append / x.extend`	add one / many
`x.insert(i,value)`	insert by index
`x.remove(value)`	remove first match
`x.pop(i)`	remove and return
`x.sort()`	mutate; returns None
`y = sorted(x)`	new sorted list
`y = x.copy()`	independent shallow copy

Tuple patterns

`p = (3.0,4.0)`	create pair
`one = (3.0,)`	singleton comma
`x,y = p`	unpack pair
`low,high = high,low`	swap
`return a,b`	return tuple
`for a,b in records`	unpack loop
`list(p), tuple(x)`	convert

Suggested reading and source basis

- Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026, Session 5.
- Y. Wang, *Programming for Engineers with Python*, working draft, August 2026, Chapter 5.
- J. V. Guttag, *Introduction to Computation and Programming Using Python*, 2nd ed., Chapter 5, Sections 5.1 and 5.3–5.5.

Preparation habit

Predict indices and slices, trace every mutation, preserve raw data, and justify list versus tuple from engineering intent.

NAZARBAYEV UNIVERSITY

A collection is not merely several values in brackets

Its order, mutability, units, and structure express an engineering decision. Choose the representation deliberately, then access, change, and verify it without losing meaning.

ENG 101 - Programming for Engineers